CUDA port of the ECCp-131 and ECC2-131 rho walks

Study `research/certicom131_cuda_port_20261023/`, built on the merge commit `6dc7efecd`. Run window 2026-10-23. Follow-up to the CPU parity study (`research/certicom131_rho_parity_20261023/`). Every number is labelled **measured**, **counted** or **derived**; raw records are in `raw/`, every value in `results.json`.

Summary

- **What exists.** Both walks are written once as per-thread functions (`gpu_walk.cuh`) that compile as CUDA device code and also run unchanged on the CPU. A launcher (`walk_cuda.cu`) holds the two kernels, a device parity gate and a timed launch. The CUDA program compiles (host and device) with clang 18 and assembles with `ptxas` 12.9 for sm_80, sm_86, sm_89 and sm_90.
- **What was NOT run: anything on a GPU.** This container has no GPU and no `nvcc`; the earlier approved cloud GPU job was rejected for a zero GPU quota (`problems/certicom-binary-gpu-rho-integration.md`). So there is **no device parity result and no GPU rate**. The launcher will produce both on the first CUDA machine; until then the port is "compiles and is correct on the CPU", not "works on a GPU".
- **Measured (CPU run of the identical per-thread code, all gates pass).** Field oracle: 0 mismatches (ECCp-131: 20,000 products and 200 inversions against the header's Montgomery code; ECC2-131: 1,280 products, 128 squares, 128 inversions of the 32-lane bitsliced field against the header's carry-less code). Step oracle: all 64 prime and all 1,024 binary lanes equal plus or minus the header's affine add after one iteration. All end points on the curve. End states identical across four splits of the lanes into threads and batches. Zero exceptional (zero-denominator) lanes. Repeated at 200 prime and 40 binary iterations.
- **Measured (static, `ptxas -v`).** ECCp-131 kernel: 64 registers per thread (84 for the retargeted sm_120 build), 1,992 bytes of stack, no spills. ECC2-131 kernel: 255 registers (the cap), about 28.5 KB of stack per thread, with 116 to 140 bytes of spill stores inside the thread function. These are compile facts, not speed; the binary kernel's local-memory footprint is the first thing to fix.
- **Classification (AGENTS.md section 3).** Implementation work; no speed claim, no improvement claim, no leaderboard or `benchmarks/README.md` row.

The two ports

ECCp-131 (`prime_thread`): each thread advances B walks (up to 16). Field element: 5 limbs of 32 bits in the header's Montgomery form (R = 2^160); interleaved (CIOS) multiply written with 32x32-to-64 multiply-accumulates. The sm_90 PTX expresses these as `mul.wide.u32` plus 64-bit adds (103 `mul.wide.u32`, no `mad` instructions in the PTX); whether `ptxas` fuses them into multiply-add in the final machine code was not inspected, and hand-written PTX carry chains (`mad.lo.cc`/`madc`) were not tried. One Fermat inversion per batch by Montgomery's trick, negation map (class representative: the one of y and p minus y whose Montgomery limb 0 is even), 16-entry r-adding table passed by value as a 640-byte kernel parameter. State is struct-of-arrays in global memory (word k of lane g at `S[k*L + g]`) so consecutive threads read consecutive words.

ECC2-131 (`bin_thread`): each thread advances 32 walks per slot, one bit lane each, S slots (up to 8), on the ECC2K-130 client's own bitsliced field code (`ecc2k130/include/fieldbs.h`, generated `eccF131.h`) instantiated with a 32-bit word. The curve constants, the jump table and the start points are moved once into the client's normal-basis coordinates through its generated change-of-basis table; multiplication, squaring (a free index permutation) and the Itoh-Tsujii inversion are the client's. The 16-way table select is a minterm-mask multiplexer over lane masks held in global memory; one inversion per batch of S slots; class representative: of (x, y) and (x, x + y) keep the one with y at the first set coordinate of x equal to 0. Coalescing: word i of slot s for thread t at `X[(s*131 + i)*nthreads + t]`.

Both walks omit jump-scalar bookkeeping and the distinguished-point test (unpriced work, as in the CPU studies). The host programs build the start points, tables and layouts from the header's verified affine arithmetic (`host_common.hpp`).

Evidence

Gates (`host_emul.cpp`; the per-thread functions called thread by thread; the GPU kernels differ only in the thread index source and the launch). G1 field oracle, G2 step oracle, G3 on-curve, G4 independence from the split of lanes into threads and batches (prime: 64 lanes as 64x1, 16x4, 8x8, 4x16 threads x batch; binary: 1,024 lanes as 32x1, 16x2, 8x4, 4x8 threads x slots). Two runs: 12 prime and 6 binary iterations, and 200 and 40 (`raw/host_emul.stdout`, `raw/host_emul_long.stdout`). Both: all gates pass, every count zero.

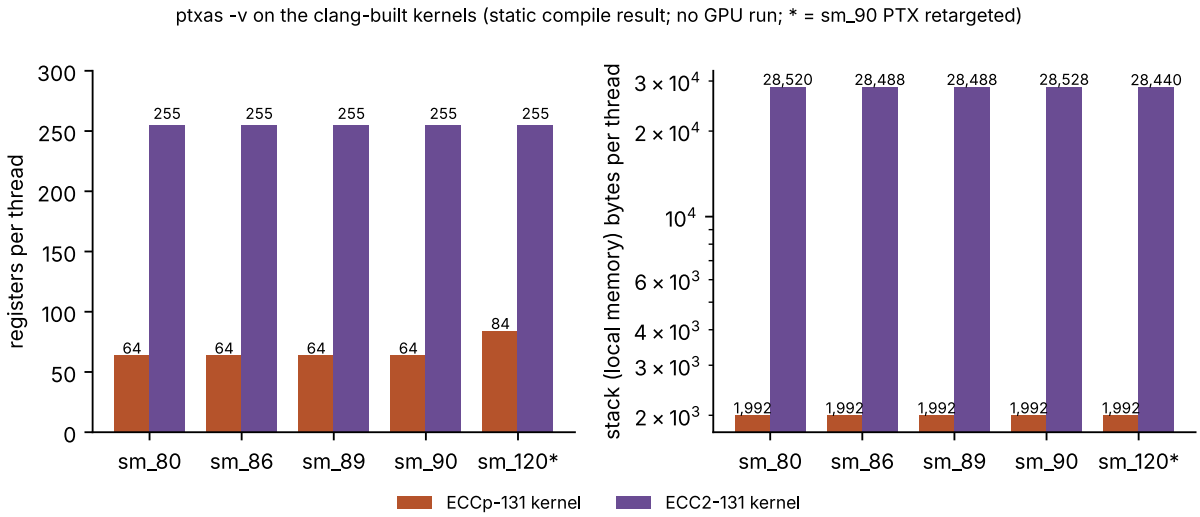


Figure 1: Measured by `ptxas -v` (CUDA 12.9) on the clang 18 builds; static compile statistics, no GPU run. sm_80 (A100), sm_86, sm_89 (Ada) and sm_90 are native clang builds. sm_120 (Blackwell, the RTX PRO 6000 class) is the sm_90 PTX with its `.version` and `.target` lines edited to 8.7 and sm_120, because clang 18 cannot emit sm_120; it shows what the Blackwell register allocator does with this code, and is not a native compile.

kernel	arch	registers	stack bytes	spill stores in the thread function
ECCp-131	sm_80 / 86 / 89 / 90	64	1,992	0
ECCp-131	sm_120 (retargeted)	84	1,992	0
ECC2-131	sm_80 / 86 / 89	255	28,520 / 28,488 / 28,488	116
ECC2-131	sm_90	255	28,528	132
ECC2-131	sm_120 (retargeted)	255	28,440	140

Table 1: Measured by `ptxas -v`. The kernel entry itself reports 0 spill bytes everywhere; the spills are in the called `bin_thread` function (its own record). Stack is per-thread local memory.

Tool versions (recorded): clang 18.1.3 (front end), `ptxas` CUDA 12.9.86. CUDA headers came from the `nvidia-*-cu12` pip wheels, as the repository's `check-cuda` recipe describes. `nvcc` is not in those wheels, so an `nvcc` build of the same source is untested.

What this does not show, and the risks

- No kernel ran on a GPU. Device parity (device end state equal to the CPU end state word for word) and the timed launch exist in `walk_cuda.cu` but have never executed. Unexecuted risks: GPU-specific behaviour of the local-memory arrays and the `__noinline__` field functions, an `nvcc`-only compile problem, and any difference between device and host integer semantics (I expect none: every operation is 32-bit integer or logic).
- No speed claim. The binary kernel is a direct port: 255 registers and 28 KB of stack per thread means almost all of its working set lives in local memory, so I expect it to be far from the ECC2K-130 packed kernel (12.5 to 17.1 B updates/s on the recorded cards). I did not estimate a rate and will not until one is measured.
- The CPU gates validate the algorithm and the data layout, not the PTX. A mismatch that only appears in compiled device code would show up in the device parity gate.
- The binary arm uses the client's generic bitsliced code with its generic multiplier, not its tuned packed kernels (carry-less multiply, block inversion, shared-memory tricks); porting those is the main speed work.

How to run it on a GPU machine

`make cuda-run` in this directory (needs a CUDA toolkit with `nvcc`; set `SMARCH`, for example `sm_120`). It prints `device_parity` lines (the gate: `words_differing` must be 0 and `exceptional` 0) and `device_rate` lines (one timed launch each, not a campaign rate), and exits nonzero on any mismatch. `make host-test` re-runs the CPU gates; `make cuda-compile` repeats the offline compile (set `CUDAPATH` to a tree with the headers and `bin/ptxas`).

Next steps

1. Run `walk_cuda` on the first CUDA machine (the open task in `problems/certicom-binary-gpu-rho-integration.md`) and keep its device parity and rate lines as the first evidence for both walks.
2. Binary kernel: move `den` and `pre` out of local arrays, cut the 255-register footprint, then port the client's packed kernel structure; measure against the recorded ECC2K-130 rates on the same card.
3. Prime kernel: tune B and the block size, test hand-written PTX multiply-add carry chains against the compiler's lowering (inspect the SASS first), and measure against a batched CPU baseline on the same inputs.
4. Add the distinguished-point test and jump-scalar bookkeeping to both kernels so the unpriced work is priced.

Files

`gpu_walk.cuh`, `host_common.hpp`, `host_emul.cpp`, `walk_cuda.cu`, `Makefile`, `run_port.sh` (frozen command list), `analyze_port.py`, `results.json`, `fig1_ptxas.svg/png`, `raw/` (gate outputs, compile logs, `ptxas -v` outputs, tool versions, source hashes, `exit_codes.txt`).